

ZenoDEX Full-System Appendix: Public Artifact Map and Claim Ledger

Dana Edwards

April 2026

1 Purpose

This appendix is the audit companion to the main public whitepaper. Its job is to pin each major public claim family to concrete repo-local artifacts.

The governing evidence rule is

`proved > contract > implemented > replay_backed > hypothesis.`

Standard reading: the tables below are claim-to-artifact maps, not marketing summaries. Practical consequence: a reviewer can audit the public package without relying on non-public development notes or unpublished implementation tooling details.

2 Spot theorem map

Claim family	Evidence class	Primary public artifacts
Canonical batch winner	proved	<code>lean-mathlib/Proofs/BatchAuctionCanonical.lean</code> ; related batch objective files under <code>lean-mathlib/Proofs/</code>
Exact-in canonical winner shell	proved	<code>lean-mathlib/Proofs/ZenoDEXExactInTrueKeyWinner.lean</code> ; route certificate companion files under <code>lean-mathlib/Proofs/</code>
CPMM reserve and K safety	proved	<code>lean-mathlib/Proofs/CPMMInvariants.lean</code> ; supporting settlement algebra under <code>lean-mathlib/Proofs/CPMMSettlement.lean</code>
Fee conservation with dust carry	proved	<code>lean-mathlib/Proofs/FeeDustCarryConservation.lean</code> ; <code>lean-mathlib/Proofs/FeeAwareBatchKGap.lean</code>
Order noncommutativity boundary	proved	<code>lean-mathlib/Proofs/OppositeDirectionNoncommutativity.lean</code>

Table 1: Main theorem-bearing spot artifacts.

3 Settlement and certificate map

Surface	Evidence class	Primary public artifacts
End-to-end settlement packet	contract + proved helper shells	<code>src/integration/settlement_end_to_end_certificate_packet.py</code> ; packet kernel under <code>src/kernels/dex/</code> ; supporting Lean packet proofs
Settlement strong certificate	contract + tests	<code>src/integration/settlement_strong_certificate.py</code> ; integration tests under <code>tests/integration/</code>
Spot / LP value contracts	contract	<code>src/integration/settlement_value_contract.py</code> ; <code>src/integration/settlement_lp_value_contract.py</code>
Price provenance and attestation	contract + tests	<code>src/integration/settlement_price_attestation.py</code> ; <code>src/integration/settlement_price_provenance.py</code> ; packet tests under <code>tests/integration/</code>
Quote receipt verification	contract + tests	<code>src/core/quote_receipts.py</code> ; receipt gates under <code>src/kernels/dex/quote_receipt_*</code> ; integration tests under <code>tests/integration/</code>

Table 2: Settlement and certificate surfaces.

4 Runtime, release, and replay map

5 Risk and monetary map

6 Reference algorithm skeletons

A. Canonical bounded route selection.

$$\text{Winner}(C) := \arg \min_{c \in C} \text{Key}(c).$$

Standard reading: once a bounded candidate family is emitted, the chosen route is the unique minimum under the explicit total key. Practical consequence: public runtime selectors should be audited as certificate-carrying argmin procedures rather than loop-order heuristics.

Operational skeleton:

Surface	Evidence class	Primary public artifacts
Kernel composition contract	contract	docs/KERNEL_ABI_AND_COMPOSITION.md; src/kernels/dex/zenodex_system_compose_v2.yaml
Public RC1 surface matrix	contract	docs/RC1_VERIFIED_SURFACE_MATRIX.md; docs/RC1_SCOPE.md
Public replay contract	contract + replay evidence	docs/PUBLIC_ASSURANCE_REPLAY.md; tools/permissionless_assurance.py
Bounded TLA claim surface	public formal claim	docs/TLA_CLAIM_SUMMARY.md; tools/run_tla_models.py
Supported HTTP boundary	contract	docs/RC1_SUPPORTED_RUNTIME_PATH.md; src/integration/api_server.py
Snapshot recovery and deterministic serialization	implementation + tests	src/integration/dex_snapshot.py; tests/integration/test_dex_snapshot.py

Table 3: Runtime and release-boundary artifacts.

Surface	Evidence class	Primary public artifacts
Perp epoch safety posture	mixed: contract, kernels, replay evidence	docs/derivatives/PERP_EPOCH_SAFETY_V1.md; perps kernels under src/kernels/dex/; src/integration/perp_engine.py
Funding witness public settlement lane	contract	public funding witness kernels and replay notes under docs/PUBLIC_ASSURANCE_REPLAY.md
ZUSD balance-law helper theorems	mixed with strong helper proofs	lean-mathlib/Proofs/ZUSDualConservation.lean; lean-mathlib/Proofs/ZUSDCollateralFlowAlgebra.lean; lean-mathlib/Proofs/ZUSDDebtHomomorphism.lean
ZUSD oracle safety contract	contract + tests	src/integration/zusd_oracle_contracts.py; tests/integration/test_zusd_oracle_contracts.py

Table 4: Risk and monetary artifacts.

1. emit a bounded candidate family C from the current admitted route domain;
2. compute the explicit runtime key for each candidate;
3. select $w = \text{Winner}(C)$;
4. build the route certificate or packet around w ;
5. expose the result as canonical only when the guarded packet verifies.

B. Settlement packet composition.

$$\text{packet_ok} \leftrightarrow \text{strong_certificate_ok} \wedge \text{value_lane_ok} \wedge \text{provenance_ok}.$$

Standard reading: an end-to-end settlement packet is accepted only when each constituent lane verifies and the bundle remains coherent. Practical consequence: settlement bundling is fail-closed composition, not best-effort aggregation.

Operational skeleton:

1. build the strong settlement certificate on the canonical delta table;
2. attach value-lane and provenance packet material;
3. serialize the combined packet;
4. verify the packet by re-checking each component gate and the bundle-level shell.

C. Runtime release replay.

$$\text{ReleaseClaimOK} := \text{CleanTree} \wedge \text{ScopeFrozen} \wedge \text{ReplayGreen} \wedge \text{ExclusionsHonest}.$$

Standard reading: the public release claim is valid only on a clean tree with a frozen documented scope and green replay lanes. Practical consequence: the release process is itself a public artifact, not only a private operator habit.

Primary public replay entry points are:

- `python3tools/permissionless_assurance.pystatus`
- `python3tools/permissionless_assurance.pyreplaypublic`
- `python3tools/permissionless_assurance.pyreplaycritical`
- `python3tools/permissionless_assurance.pyreplayfull`

7 How to read the claim boundary

The public package uses a three-level distinction:

$$\text{FullTargetArchitecture} \supset \text{CurrentPublicEvidenceFrontier} \supset \text{CurrentRC1Claim}.$$

Standard reading: the complete intended architecture is broader than the public evidence frontier, and the evidence frontier is broader than the current RC1 public claim. Practical consequence: the appendix helps reviewers see exactly which artifacts support which class of claim today.